



Republic of the Philippines

BATANGAS STATE UNIVERSITY

The National Engineering University

Lipa Campus
Marawoy, Lipa City

IT-314

Web System and Technologies

LEARNING MODULE

Prepared by:

Bryan Emmanuel B. Paz
Lecturer I

CONTENTS IN BRIEF

Course Code: IT 314
Course Title: Web System and Technologies

Module 5 Web System and Technologies

No.	Unit	Lesson Content
1	PHP: Hypertext Preprocessor Basics	• Escaping to PHP • Commenting PHP • Hello World • Variables in PHP • Statements in PHP • Loops in PHP • PHP Arrays • Functions • Connecting to a Database • PHP Form Handling • PHP Include & Require Statements

PHP: Hypertext Preprocessor: Basics

Getting Started with PHP

PHP (Recursive acronym for PHP: Hypertext Preprocessor)

Introduction

The first version of PHP was created by Rasmus Lerdorf in 1994 as a set of Common Gateway Interface (CGI) scripts. Initially, PHP stood for *Personal Home Page*, but now it stands for the recursive acronym *PHP: Hypertext Preprocessor*. The current PHP implementation is produced by **The PHP Group** and is maintained through contributions from a large community of developers.

As of 2025, **PHP 8.x** is the latest stable version, with the **PHP 9.x** branch anticipated in the coming years. PHP 8 introduced significant improvements like the Just-in-Time (JIT) compiler, better error handling, and named arguments, which have enhanced performance and developer experience.

Where is PHP Used?

PHP is highly versatile, and there are three main areas where it is primarily used:

1. Server-side Scripting

- This is the most common use of PHP. Server-side scripting means that PHP scripts are executed on the web server, and the results are sent to the browser. The PHP engine processes requests and generates dynamic content such as HTML, JavaScript, or JSON responses.
- Most PHP applications are used in conjunction with databases like MySQL, PostgreSQL, or SQLite to store and retrieve dynamic content.

2. Command Line Scripting

- PHP scripts can be run directly from the command line without a web server or browser. This is ideal for tasks like automating system jobs (cron jobs on Linux, Task Scheduler on Windows), or text-based scripts for file manipulation, system administration, etc.

3. Desktop Applications

- While not commonly used for GUI-based desktop applications, PHP can be used with libraries like **PHP-GTK** to develop cross-platform desktop applications. However, for most desktop applications, languages like Python, Java, or C# are preferred due to more robust frameworks for GUI.

Why PHP?

There are several compelling reasons why PHP continues to be one of the most widely used languages for web development:

Compatibility with Servers

PHP is compatible with almost all web servers, including:

- **Apache (most common)**
- **Nginx**
- **IIS**
- **LiteSpeed**
- **Cherokee**
- **And more...**

This cross-server compatibility makes it highly versatile, allowing it to run on virtually any server environment.

2. Platform Independence

PHP is a cross-platform language, meaning it can run on **Windows, macOS, Linux, Unix**, and more. This is particularly advantageous for developers who work in heterogeneous environments and need their code to run across different systems.

3. Extensive Database Support

PHP supports a wide range of databases, which is one of the reasons it is so popular for web development. It can easily interact with databases like:

- **MySQL**
- **PostgreSQL**
- **SQLite**
- **MongoDB**
- **Oracle DB**
- **Microsoft SQL Server (MSSQL)**
- **MariaDB**
- **And more...**

This broad database compatibility is key for creating dynamic websites that need to store and manage data efficiently.

4. Open Source & Free

PHP is open-source and free to download from the official site (php.net). It's maintained by a large community of developers, and anyone can contribute to its improvement. This open-source nature has led to the widespread adoption of PHP, especially in small to medium-sized businesses and startups that need a cost-effective solution for web development.

5. Large Community & Ecosystem

One of PHP's strongest points is its vast and active community. With a massive number of online forums, tutorials, and resources, PHP developers can get help from peers quickly. There are also countless libraries and frameworks (like **Laravel, Symfony, CodeIgniter, Zend Framework**) that help developers build secure, high-quality web applications faster and more efficiently.

6. Easy to Learn

PHP is a relatively easy language for beginners to learn. Its syntax is simple and similar to other C-based languages, which makes it easy for developers familiar with other languages to get started. The low barrier to entry, combined with its massive online community, makes PHP a popular choice for both beginners and experienced developers.

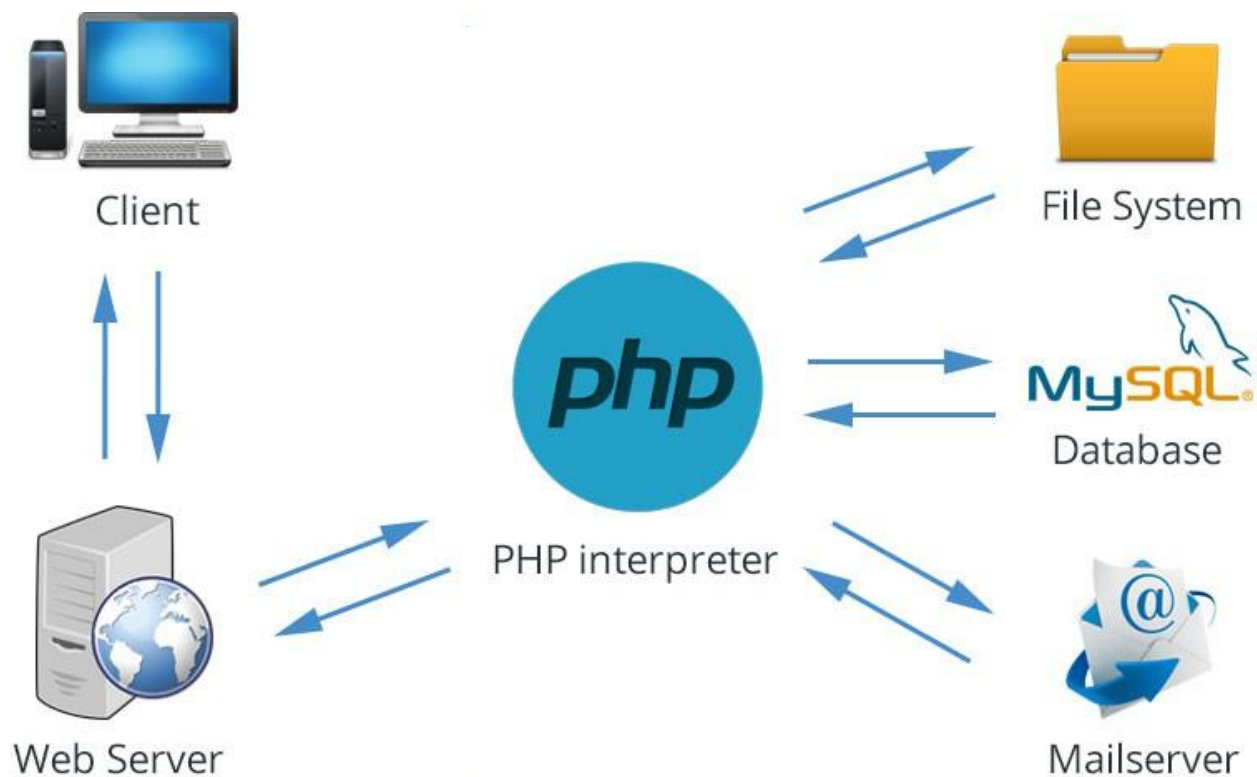


Figure 1.1: PHP Dynamic Content Interaction

XAMPP Setup

XAMPP is a free and open source cross-platform web server solution developed by Apache Friends, consisting mainly of the Apache HTTP Server, MariaDB database, and interpreters for scripts written in the PHP and Perl programming languages. In order to make your PHP code execute locally, first install XAMPP.

- [Download XAMPP](#)
- Install the program (check the technologies you want during installation)

Open XAMPP and click on "Start" on Apache and MySQL (when working with databases)

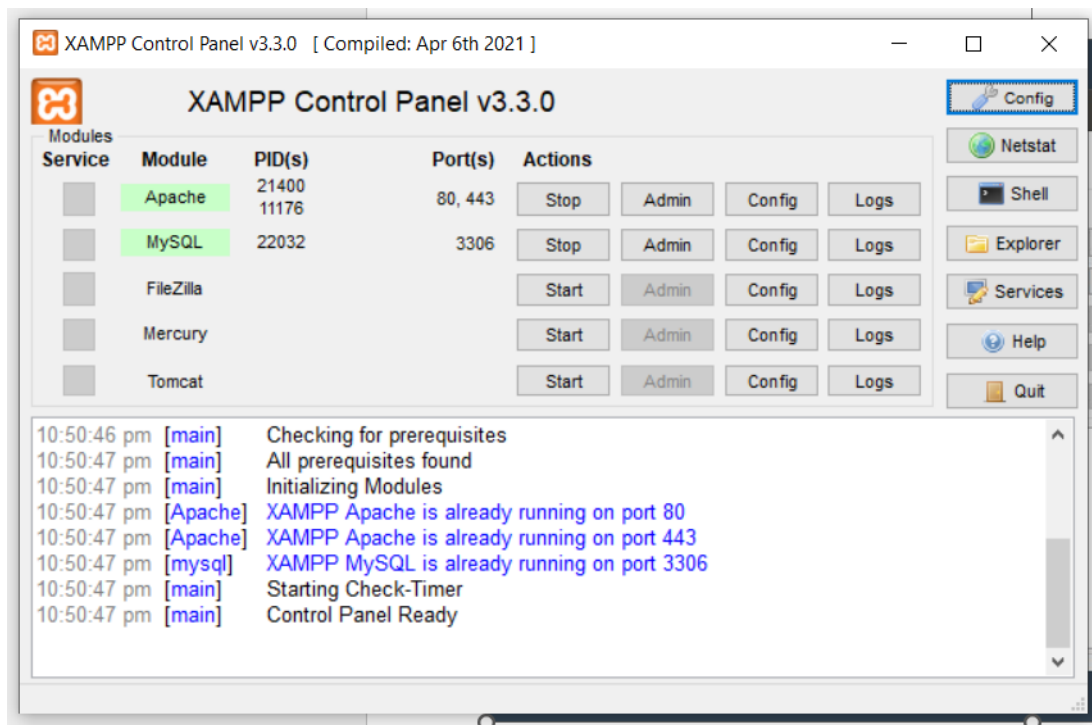


Figure 1.2: XAMPP window after a successful installation with Apache and MySQL enabled

- Place your web project inside the `htdocs` directory. In the common case, if you installed XAMPP directly inside the C: drive of your PC, the path to this folder would be: `C:xampphtdocs`
- To test the services are up and running you can just enter **localhost** in your address bar and expect the welcoming page.

PHP Language Basics

The aim of this section is to introduce the general syntax you should expect in PHP.

Escaping to PHP

There are four ways the PHP parser engine can differentiate PHP code in a webpage:

Canonical PHP Tags

This is the most popular and effective PHP tag style and looks like this:

```
<?php...?>
```

Short-open Tags

These are the shortest option, but they might need a bit of configuration, and you might either choose the `--enable-short-tags` configuration option when building PHP, or set the

```
<?...?>
```

`short_open_tag` setting in your `php.ini` file.

ASP-like Tags

In order to use ASP-like tags, you'll need to set the configuration option in the `php.ini` file:

```
<%...%>
```

HTML script Tags

You can define a new script with an attribute language like so:

```
<script language="PHP">...</script>
```

Commenting on PHP

Just like other languages, there are several ways to comment PHP code. Let's have a look at the most useful ones: Use `#` to write single-line comments

```
<?
    # this is a comment in PHP, a single line comment
?>
```

Use `//` to also write single-line comments

```
<?
    // this is also a comment in PHP, a single line comment
?>
```

Use `/* ... */` to write multi-line comments

```
<?
    /* this is a multi line
       comment Name: Web Code
       Geeks
       Type: Website
       Purpose: Web Development
    */
?>
```

Hello World

The very basic example of outputting a text in PHP would be:

```
<?
print("Hello
World"); echo
"Hello World";
printf("Hello
World");
```

The result of the above statements would be the same: "Hello World". But why are there three different ways to output?

- **print** returns a value. It always returns 1.
- **echo** can take a comma delimited list of arguments to output.
- **printf** is a direct analog of C's `printf()`.

Variables in PHP

Any type of variable in PHP starts with a leading dollar sign (\$) and is assigned a variable type using the = (equals) sign. The value of a variable is the value of its most recent assignment. In PHP, variables do not need to be declared before assignment. The main data types used to construct variables are:

- **Integers** - whole numbers like 23, 1254, 964 etc
- **Doubles** - floating-point numbers like 46.2, 733.21 etc
- **Booleans** - only two possible values, true or false
- **Strings** - set of characters, like *Web Code Geeks*
- **Arrays** - named and indexed collections of other values
- **Objects** - instances of predefined classes

The following snippet shows all of these data types declared as variables:

```

2
<?
    $intNum = 472;
    $doubleNum = 29.3;
    $boolean = true;
    $string = 'Web Code Geeks';
    $array = array("Pineapple", "Grapefruit", "Banana");

    // creating a class before declaring an object
    variable class person {
        function agePrint() {
            $age = 5;
            echo "This person is $age years old!";
        }
    }
    // creating a new object of type person
    $object = new person;
?>

```

Conditional Statements in PHP

Conditional statements are used to execute different code based on different conditions.

The If statement

The **if** statement executes a piece of code if a condition is true. The syntax is:

```

if (condition) {
    // code to be executed in case the condition is true
}

```

A practical example would be:

```

<?php
    $age = 18;

    if ($age < 20) {
        echo "You are a teenager";
    }
?>

```

Because the condition is true, the result would be:

```

You are a teenager

```

The If... Else statement

The If... Else statement executed a piece of code if a condition is true and another piece of code if the condition is false. The syntax is:

```
if (condition) {  
    // code to be executed in case the condition is true  
}  
else {  
    // code to be executed in case the condition is false  
}
```

An example of an If... Else statement would be:

```
<?php  
$age = 25;  
  
if ($age < 20) {  
    echo "You are a teenager";  
}  
else {  
    echo "You are an adult";  
}  
?>
```

Because the condition is false, the result in this case would be:

```
You are an adult
```

The If... Elseif... Else statement

This kind of statement is used to define what should be executed in the case when two or more conditions are present. The syntax of this case would be:

```
if (condition1) {  
    // code to be executed in case condition1 is true  
} elseif (condition2) {  
    // code to be executed in case condition2 is true  
}  
else {  
    // code to be executed in case all conditions are false  
}
```

Again, a simple example to demonstrate this:

```
<?php  
$age = 3;  
  
if ($age < 10) {  
    echo "You are a kid";  
} elseif ($age < 20) {  
    echo "You are a teenager";  
} else {  
    echo "You are an adult";  
}  
?>
```

The result, as you might expect, would be:

```
You are a kid
```

Loops in PHP

In PHP, just like any other programming language, loops are used to execute the same code block for a specified number of times. Except for the common loop types (for, while, do . . . while), PHP also support `foreach` loops, which is not only specific to PHP. Languages like Javascript and C# already use **foreach** loops. Let's have a closer look at how each of the loop types works.

The for loop

The `for` loop is used when the programmer knows in advance how many times the block of code should be executed. This is the most common type of loop encountered in almost every programming language.

```
for (initialization; condition; step){  
    // executable code  
}
```

An example where we use the for loop would be:

```
for ($i=0; $i < 5; $i++) {  
    echo "This is loop number $i";  
}
```

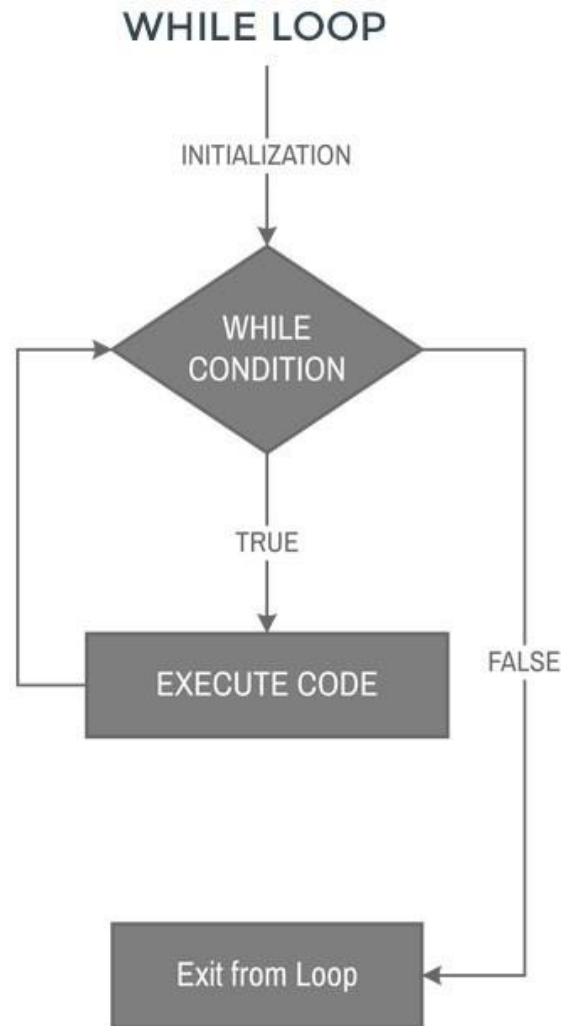
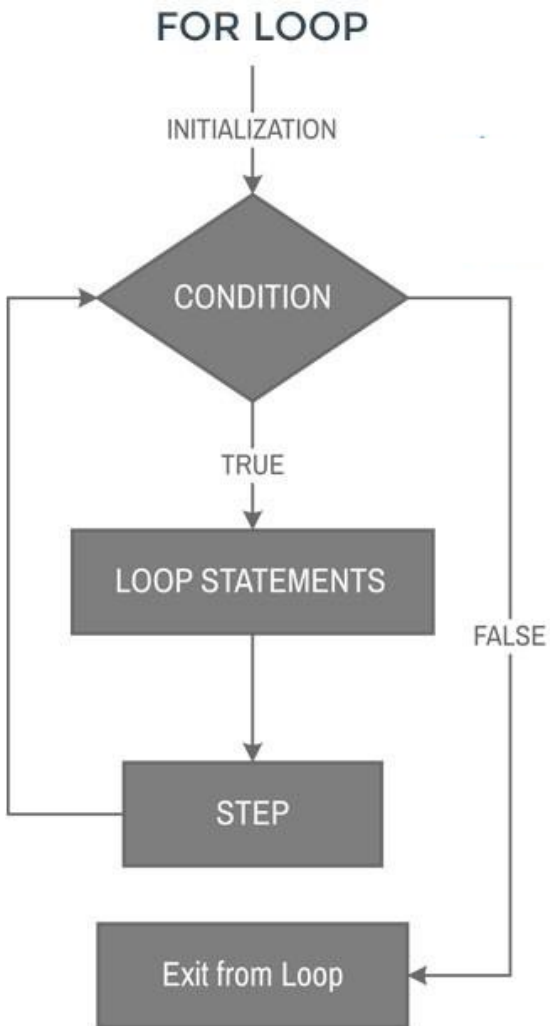
The while loop

The `while` loop is used when we want to execute a block of code as long as a test expression continues to be true.

```
while (condition){  
    // executable code  
}
```

An example where we use the while loop would be:

```
$i=0; //  
    initialization  
while ($i < 5) {  
    echo "This is loop number $i";  
    $i++; // step  
}
```



The do...while loop

The do...while loop is used when we want to execute a block of code at least once and then as long as a test expression is true.

```

do {
    // executable code
}
while (condition);
  
```

An example where we use the do...while loop would be:

```

$i = 0; //
initialization do {
    $i++; // step
    echo "This is loop number $i";
}
while ($i < 5); // condition
  
```

The foreach loop

The `foreach` loop is used to loop through arrays, using a logic where for each pass, the array element is considered a value and the array pointer is advanced by one, so that the next element can be processed.

```
foreach (array as value) {  
    // executable code  
}
```

An example where we use the `foreach` loop would be:

```
$var = array('a','b','c','d','e'); // array declaration  
  
foreach ($var as $key) {  
    echo "Element is  
    $key";  
}
```

PHP Arrays

Arrays are used to store multiple values in a single variable. A simple example of an array in PHP would be:

```
<?php  
$languages = array("JS", "PHP", "ASP", "Java");  
?>
```

Array elements are accessed like this: `$arrayName[positionIndex]`. For the above example we could access "PHP" this way: `$languages[1]`. Position index is 1 because in programming languages the first element is always element 0. So, PHP would be 1 in this case.

There are three types of arrays in PHP:

Indexed Arrays

We can create these kind of arrays in two ways shown below:

```
<?php  
$names = array("Fabio", "Klevi", "John");  
?>
```

```
<?php  
// this is a rather manual way of doing it  
$names[0] = "Fabio";  
$names[1] = "Klevi";  
$names[2] = "John";  
?>
```

An example where we print values from the array is:

```
<?php  
$names = array("Fabio", "Klevi", "John");  
echo "My friends are " . $names[0] . ", " . $names[1] . " and " . $names[2];  
?>
```

Looping through an indexed array is done like so:

```
<?php
    $names = array("Fabio", "Klevi", "John");
    $arrayLength = count($names);

    for($i = 0; $i < $arrayLength;
        $i++) { echo $names[$i];
        echo " ";
    }
?>
```

This would just print the values of the array.

Associative Arrays

Associative arrays are arrays which use named keys that you assign. Again, there are two ways we can create them:

```
<?php
    $namesAge = array("Fabio"=>"20", "Klevi"=>"16", "John"=>"43");
?>
```

```
<?php
    // this is a rather manual way of doing it
    $namesAge['Fabio'] = "20";
    $namesAge['Klevi'] = "16";
    $namesAge['John'] = "43";
?>
```

An example where we print values from the array is:

```
<?php
    $namesAge = array("Fabio"=>"20", "Klevi"=>"16",
    "John"=>"43"); echo "Fabio's age is " .
    $namesAge['Fabio'] . " years old.";
```

```
// RESULT
Fabio's age is 20 years old.
```

Looping through an associative array is done like so:

```
<?php
    $namesAge = array("Fabio"=>"20", "Klevi"=>"16", "John"=>"43");

    foreach($namesAge as $i => $value) {
        echo "Key = " . $i . ", Value = " .
        $value; echo " ";
    }
?>
```

Multidimensional Arrays

This is a rather advanced PHP stuff, but for the sake of this tutorial, just understand what a multidimensional array is. Basically, it is an arrays the elements of which are other arrays. For example, a three-dimensional array is an array of arrays of arrays. An example of this kind of array would be:

```
<?php
    $socialNetowrks = array (
        array("Facebook", "feb", 21),
        array("Twitter", "dec", 2),
        array("Instagram", "aug", 15));
?>
```

PHP Functions

Functions are a type of procedure or routine that gets executed whenever some other code block calls it. PHP has over 1000 built-in functions. These functions help developers avoid redundant work and focus more on the logic rather than routine things. Apart from its own functions, PHP also let's you create your own functions. The basic syntax of a function that we create is:

```
<?php
function functionName($argument1, $argument2...) {
    // code to be executed
}

functionName($argument1, $argument2...); // function call
?>
```

Every function needs a name, optionally has one or more arguments and most importantly, defines some kind of procedure to be followed within the body, that is, code to be executed. Let's see some basic functions:

```
<?php
function showGreeting() { // function definition
    echo "Hello Chloe!"; // what this function
    does
}

showGreeting(); // function call
?>
```

This function would just print the message we wrote:

```
Hello Chloe!
```

Another example would be a function with arguments:

```
<?php
function greetPerson($name) { // function definition with
    arguments echo "Hi there, ".$name;    // what this function
    does
}

greetPerson("Fabio"); // function call
greetPerson("Michael");
?>
```

This time we'd have:

```
Hi there,
Fabio Hi
..
```

More than one argument can be used whenever needed:

```
<?php
function personProfile($name, $city, $job) { // function definition with
    arguments echo "This person is ".$name." from ".$city.".";
    echo "";
    echo "His/Her job is ".$job.".";
}

personProfile("Fabio", "Tirana", "Web Dev");
echo "";
personProfile("Michael", "Athens", "Graphic
Designer"); echo "";
personProfile("Xena", "London", "Tailor");
?>
```

The result would include the printed message together with the arguments:

```
This person is Fabio from Tirana.
His/Her job is Web Dev.
This person is Michael from
Athens. His/Her job is Graphic
Designer.
This person is Xena from London.
His/Her job is Tailor.
```

In PHP, just like in many other languages, we can tell functions to return a value upon executing the code. Such example would be:

```
<?php
function difference($a, $b) { // function definition with arguments
    $c = $a -
    $b; return
    $c;
}

echo "The difference of the given numbers is: ".difference(8, 3);
?>
```

The result would be:

```
The difference of the given numbers is: 5
```

Connecting to a Database

There are four ways you can generally consider when you want to connect to a previously created database. Below, we'll explain how you can use each of them beginning with the easiest one.

Connecting to MySQL Databases

The syntax for connecting to a MySQL database would be:

```
<?php
$username = "your_name";
$password = "your_password";
$hostname = "localhost";

//connection to the database
$dbConnect = mysql_connect($hostname, $username,
$password) or die("Unable to connect to MySQL");
echo "Connected to MySQL";

//select a specific database
$dbSelect = mysql_select_db("dbName",
$dbConnect) or die("Could not select dbName");
?>
```

Considering your entered information is correct, you'd be successfully connected to the right database and ready to start writing and test your queries. Else, the respective error message would appear as defined by the `die` function. However, do keep in mind that the `mysql` extension is deprecated and will be removed in the future, so the next methods can be used for databases.

Connecting to MySQLi Databases (Procedural)

The MySQLi stands for MySQL improved. The syntax for connecting to a database using MySQLi extension is:

```
<?php
$username = "your_name";
$password = "your_password";
$hostname = "localhost";

//connection to the database
$dbConnect = mysqli_connect($hostname, $username, $password)

// another way of checking if the connection was
successful if (!$dbConnect) {
    die ("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";

//select a specific database
mysqli_select_db($dbConnect,
"dbName")
~
```

This is a good way to start, because it is easy to understand and gets the job done. However, object oriented logic that we'll see below is what everyone should be getting into because of the other components of programming being used in this paradigm and also because it is kind of more structured way of doing things.

PHP Form Handling

In HTML, forms are used to collect user input/information. But, as far as HTML goes, it just provides the graphical interface and the ability to write on the input fields. However, the aim we collect user information is because we need to process this information. That usually means saving the inputs to a database so that they can be accessed later for several purposes. In this section, we will not send information to the database, as that requires an active database that is already set up and includes knowledge from SQL language, but we can retrieve information that the user has given us. That being said, it is important to have a way to get information, because what you use it for and where you want to show/save it depends. Let's have a look at the HTML form below:

```
<form method="POST" action="wcg.php">
  <input type="text" name="name" placeholder="Name">
  <input type="email" name="email" placeholder="E-Mail">
  <input type="number" name="age" placeholder="Age">
  <input type="radio" name="gender" value="Male"> Male
  <input type="radio" name="gender" value="Female"> Female
  <input type="submit" name="submit" value="Submit">
</form>
```

This form is just a regular one, and includes inputs for name, e-mail, age and gender. This information will be subject of a **print** when the Submit button is clicked. That just proves that we got the information from the user. For now, let's see what we got and fill the form:

Next, we check each input to make sure the user has written/chosen something, and the input is not empty. We do this using two well-known functions in PHP, the `isset()` and `empty()`. After making sure we have this right, we can also validate fields. In this case, only the name input may need some sort of validation to make sure one simply doesn't write numbers in there, while other inputs are more or less validated from HTML5, in the case of email and age.

```
<?php
if (isset($_POST["name"]) && !empty($_POST["name"])) {
    $name = $_POST["name"];
    if (!preg_match("/^[a-zA-Z ]*$/", $name))
        echo "Name: Only letters and whitespace
        allowed"; else
        echo "Name: " . $_POST["name"] . " ";
}
if (isset($_POST["email"]) && !empty($_POST["email"])) {
    echo "E-Mail: " . $_POST["email"] . " ";
}
if (isset($_POST["age"]) && !empty($_POST["age"])) {
    echo "Age: " . $_POST["age"] . " ";
}
if (isset($_POST["gender"]) && !empty($_POST["gender"])) {
    echo "Gender: " . $_POST["gender"] . " ";
}
?>
echo "age" . " " . $_POST["age"] . " " .
```

Validation is a must for all fields when considering real web projects

If the fields have been set, information about each field will be printed. After clicking the "Submit" button, the result we'd have is:

```
Name: Alex Brown
E-Mail:
alex.kira@gmail.com
Age: 23
Gender: Male
```

As you may have noticed by now, we used this `$_POST["name"]` to get information the user posted.

PHP Include & Require Statements

PHP code can get cluttered, which means that if you want to later change something, it becomes a hard task to do. Include and require statements are two almost identical statements that help in an important aspect of coding, the organization of code, and making it more readable and flexible. The include/require statement copies all text, code or any other markup from one existing file to the file using the statement. In a simple viewpoint, do consider these statements like this:

header.php

```
<nav>
  <ul>
    <li>Home</li>
    <li>About</li>
    <li>Contact</li>
  </ul>
</nav>
```

index.php

```
<?php include 'header.php';?>

<body>
  <h3>Page content below</h3>
  <p>Lorem ipsum dolor</p>
</body>
```

index.php

```
<nav>
  <ul>
    <li>Home</li>
    <li>About</li>
    <li>Contact</li>
  </ul>
</nav>

<body>
  <h3>Page content below</h3>
  <p>Lorem ipsum dolor</p>
</body>
```



The include and require statements are the same, except upon failure of code execution where:

- **require** will produce a fatal error (E_COMPILE_ERROR) and stop the script from executing
- **include** will only produce a warning (E_WARNING) and the script will continue

The syntax of these two statements is as follows:

```
<?php
include 'file.php'; // in case of
include require 'file.php'; // in
case of require
~
```

Let's now see a real-world example where we use a header and footer using include and require into our main file: header.php

```
<header>
<nav>
  <ul>
    <li><a href="#">Home</a></li>
    <li><a href="#">Profile</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>
</header>
// styling to make the menu look right
<style
  type="text/css">
  ul, li {
    list-style-type:
    none; display:
    inline-block;
    margin-right: 1em;
    padding: 0;
  }
</style>
```

footer.php

```
<footer>All Rights Reserved. Do not reproduce this page.</footer>
```

In our main file, we'll use require for the header.php and include for the footer.php file: index.php

```
<?php require 'header.php'; ?>

<body>
  <h2>Main Content Goes Here</h2>
</body>

<?php include 'footer.php' ?>
```